

Architektura Mikrokontrolerów

Ustrukturyzowane Podsumowanie Teoretyczne i Tryby Adresowania

1. Podstawowe Pojęcia i Mechanizmy Systemowe

Podprogram (Subroutine)

Wydzielony, autonomiczny fragment kodu realizujący określone zadanie, który może być wywoływany wielokrotnie z różnych miejsc programu głównego. Główną zaletą stosowania podprogramów jest **optymalizacja i oszczędność pamięci zajmowanej przez program**.

- **Punkt wejścia:** W języku assemblera początek podprogramu jest jednoznacznie określony przez dedykowaną etykietę.
- **Adres powrotu:** W momencie wywołania podprogramu, na stosie automatycznie zapisywany jest adres komórki pamięci zawierającej kolejną instrukcję programu głównego, do której procesor musi powrócić po zakończeniu podprogramu.

Stos (Stack)

Wydzielony obszar pamięci operacyjnej RAM przeznaczony do tymczasowego przechowywania danych, funkcjonujący w oparciu o architekturę kolejki **LIFO (Last In, First Out)** – dane zapisane jako ostatnie są odczytywane jako pierwsze.

- **Wskaźnik stosu (SP - Stack Pointer):** Rejestr procesora wskazujący na aktualny wierzchołek (szczyt) stosu.
- **Operacje podstawowe:** Zapis realizowany jest instrukcją **PUSH**, a odczyt instrukcją **POP**.
- **Dostępność danych:** Bezpośredni dostęp jest możliwy wyłącznie do elementu na szczycie stosu. Po wykonaniu odczytu (**POP**) element ten jest traktowany jako nieaktywny, a wskaźnik SP odpowiednio się przesuwa.
- **Kierunek wzrostu:** Stos może rosnąć "w dół" pamięci, co oznacza, że każdy kolejno dodawany element otrzymuje niższy adres fizyczny niż poprzedni.
- **Zastosowanie:** Przechowywanie adresów powrotu z podprogramów, obsługa przerwań oraz tymczasowe zapamiętywanie stanów rejestrów.

Wektor Przerwań (Interrupt Vector)

Komórka lub obszar w pamięci o stałym adresie fizycznym, w którym zapisany jest **adres procedury obsługi danego przerwania**.

Mechanizm działania: W momencie wystąpienia i zaakceptowania żądania przerwania, procesor natychmiastowo wstrzymuje wykonywanie bieżącego programu. Aktualna zawartość licznika rozkazów (adres instrukcji, która miała być wykonana jako następna) zostaje zabezpieczona na stosie. Następnie mikrokontroler pobiera z wektora przerwań adres docelowy i wykonuje skok do podprogramu obsługi przerwania. Po jego zakończeniu następuje powrót do odłożonego na stosie adresu.

2. Tryby Adresowania: Rodzina MC6800 / M68HC05

Tryb adresowania określa sposób, w jaki procesor interpretuje kod operacji w celu wyznaczenia efektywnego adresu operandu.

1. **Adresowanie podwójne:** Specyficzny tryb kombinowany, w którym pierwszym operandem jest domyślnie akumulator (A lub B), natomiast drugi operand jest wyznaczany za pomocą innego, standardowego trybu (natychmiastowego, bezpośredniego, rozszerzonego lub indeksowego).
2. **Akumulatorowe:** Instrukcje jednobajtowe, gdzie operand znajduje się bezpośrednio w akumulatorze A lub B.

Przykład: **INCA** (inkrementacja zawartości akumulatora A). Dla instrukcji ściągania/wkładania na stos (**PUL**, **PSH**) jest to jedyny dostępny tryb.

3. **Rejestrowe (Inherent):** Operand znajduje się wewnątrz określonego rejestru procesora, a informacja o nim jest sztywno zaszyta w samej mnemonice instrukcji.

Przykład: **INX** (inkrementacja rejestru indeksowego X). Procesor nie musi pobierać adresu z pamięci.

4. **Natychmiastowe (Immediate):** Operand (dana stała) jest bezpośrednio częścią składową instrukcji i znajduje się na kolejnych bajtach kodu operacji (2. lub 3. bajt). W asemblerze oznaczany symbolem **#**.

Przykład: **LDA #13h** — załadowanie wartości szesnastkowej 13h bezpośrednio do akumulatora A.

5. **Względne (Relative):** Adres docelowy wyznaczany jest jako przesunięcie (offset) względem aktualnej wartości licznika rozkazów (PC). Wykorzystywane głównie w instrukcjach warunkowych i skokach lokalnych. Zakres przesunięcia wynosi zazwyczaj od -128 do +127 bajtów.

6. **Indeksowe:** Adres efektywny operandu powstaje poprzez zsumowanie zawartości rejestru indeksowego (np. IX) z przesunięciem podanym w instrukcji. Oznaczane symbolem **X**.

Przykład: **5,X** — adresem komórki docelowej jest wartość w **IX + 5**.

7. **Bezpośrednie (Direct):** Drugi bajt instrukcji zawiera 8-bitowy, pełny adres komórki pamięci. Tryb ten ogranicza się do tzw. strony zerowej (adresy od 00h do FFh), gdzie starszy bajt adresu wynosi zawsze 00h, co przyspiesza wykonanie programu.

Przykład: **LDA \$13** — załadowanie do akumulatora A zawartości komórki o adresie \$0013.

8. **Bezpośrednie rozszerzone (Extended):** Instrukcja zawiera pełny, 16-bitowy adres fizyczny operandu umieszczony na drugim i trzecim bajcie instrukcji, co pozwala na dostęp do pełnego obszaru pamięci.

Przykład: **LDA \$0329** — pobranie danej z adresu komórki \$0329.

3. Tryby Adresowania: Intel 8051

A) Adresowanie wewnętrznej pamięci danych RAM (Obszar 8-bit: 00h - FFh)

- **Adresowanie bezpośrednie (Direct Addressing):** Adres komórki jest podany wprost w kodzie instrukcji. Umożliwia dostęp do pełnego obszaru wewnętrznego RAM (00h-FFh). W zakresie 0-127 (00h-7Fh) adresuje pamięć danych użytkownika, natomiast w zakresie 128-255 (80h-FFh) zapewnia wyłączny dostęp do rejestrów specjalnych (SFR).

Instrukcja: **MOV A, 32H** → Przepisanie zawartości komórki RAM o adresie 32h do akumulatora A.

- **Adresowanie natychmiastowe (Immediate Data):** Wartość stała zapisana jest bezpośrednio w pamięci programu jako część instrukcji. Poprzedzana jest znakiem **#**.

Instrukcja: **MOV A, #32H** → Wpisanie wartości stałej (liczby) 32h bezpośrednio do akumulatora A.

- **Adresowanie rejestrowe (Register Addressing):** Operacje wykonywane są bezpośrednio na rejestrach roboczych procesora (np. ACC, DPTR, lub R0 do R7 z aktualnie wybranego banku rejestrów).

Instrukcja: **MOV A, R0** → Przepisanie bieżącej wartości rejestru R0 do akumulatora A.

- **Adresowanie pośrednie (Indirect Addressing):** Rejestr wskazuje nie na samą daną, lecz na adres komórki pamięci, w której ta dana się znajduje. Oznaczone symbolem **@**. Dla wewnętrznej pamięci RAM rolę wskaźników mogą pełnić wyłącznie rejestry **R0** i **R1**, a dla pamięci zewnętrznej — 16-bitowy rejestr **DPTR**.

Instrukcja: **MOV A, @R0** → Odczytanie wartości z komórki pamięci RAM, której adres jest aktualnie zapisany w rejestrze R0, i wpisanie jej do akumulatora.

B) Pozostałe tryby specyficzne

- **Adresowanie względne (Relative Addressing):** Wykorzystywane przy instrukcjach skoków i rozgałęzień. Adres docelowy obliczany jest dynamicznie jako suma aktualnego stanu Licznika Rozkazów (PC) oraz przesunięcia (offsetu).

Mechanizm na przykładzie instrukcji **BRA \$02** :

Jeśli instrukcja skoku znajduje się pod adresem **\$1000** , to sam kod operacji zajmuje adres **\$1000** , a offset (**\$02**) zapisany jest pod **\$1001** . Licznik PC automatycznie wskazuje na adres następnej instrukcji, czyli **\$1002** . Stąd wyliczany jest skok efektywny: **$\$1002 + \$02 = \$1004$** .

- **Adresowanie bezpośrednie bitów (Bit Addressing):** Unikalna cecha architektury 8051 pozwalająca na bezpośrednie operowanie na pojedynczych bitach znajdujących się w specjalnym obszarze pamięci RAM (adresy bitowe 00h-7Fh) oraz w rejestrach SFR podzielnych przez 8.

Instrukcja: **MOV C, ACC.7** → Przepisanie wartości siódmego (najstarszego) bitu akumulatora do flagi przeniesienia (C).